Representing the current state in event sourcing is an expensive operation. You have to iterate over all the events, although there are some optimizations. A common optimization is snapshotting, where the account's events are summarized from time to time so you don't have to read the entire list, just the snapshots. But even in these cases, the model is not optimized for different queries, and organizations often need to look at the data from different angles. And every time you need to group the data differently, it's very expensive. For example, you may need the balance of each account by customer and office, then by office and country, and so on. There's no SQL you can use for that. The data is stored in events, not rows where the individual fields are arranged. For each query, you have to iterate and extract. To solve this problem, you can use a good, old-fashioned solution called a *projection*.

A projection is nothing more than a materialized view. You take the logs, group them as needed, and store them—exactly what Pat Helland said: "A database is the cache of a subset of the log." You can create a new model from the event log by reading the individual entries, grouping them appropriately, and saving them somewhere. This way, you get data that already has the form you want to query and decouple the logs from the view so they can be scaled separately. The downside is that the projection is *eventually consistent* with respect to the logs. However, since you are only using it for reading and not writing, this usually is not a problem. If you're writing, you probably need stronger consistency guarantees. This brings us to the separation of read and write, the query side and the command side.

Command Query Responsibility Segregation (CQRS; http://mng.bz/rdzj) is based on the idea of Bertrand Meyer's